DVSA Project Report

Course Project: DVSA Vulnerability Discovery and Remediation

Course / Section: ICS-344

Student Name(s) + ID(s): Omar AlJughaiman **202172810 , Team28 , Sec04**

University: King Fahd University of Petroleum and Minerals (KFUPM)

AWS Region: us-east-1

Your DVSA Website URL: http://dvsa-omar-352712712043-us-east-1.s3-website.us-east-1.amazonaws.com/store

Date: 2026-05-05

Vulnerability Title: Lesson #5: Broken Access Control (Privilege Escalation / BOLA)

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a Broken Access Control vulnerability (specifically handling of privileged actions) in DVSA. The issue exists in the `/orders` API, where the server processes privileged `admin-update` actions from standard users. Because of missing authorization enforcement inside the Lambda function, an attacker can modify the status of any order without actually holding admin rights.

Part 2) Why This Works / Root Cause

The vulnerability emerges due to a lack of proper Role-Based Access Control (RBAC) validations during the processing of the payload. The system trusts the incoming `action` field (e.g., `"action": "admin-update"`) from the request body unconditionally without checking the user's role or token privileges. This allows unauthenticated or unauthorized users to perform administrative state changes to objects.

Part 3) Environment and Setup

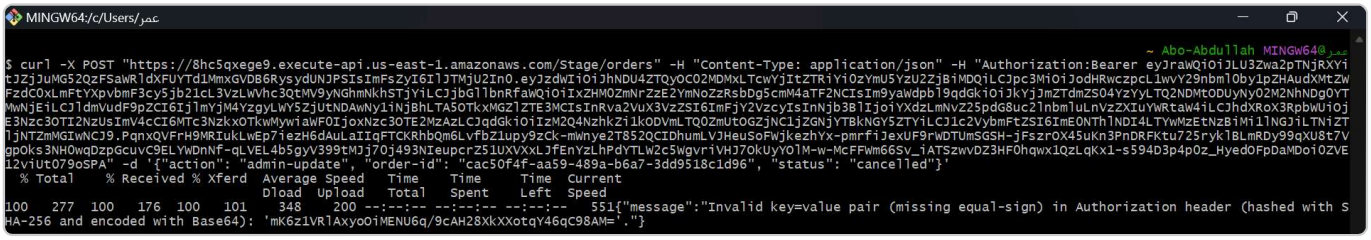
- **Target Endpoint:** The `/orders` API endpoint through API Gateway.
- **AWS Service:** AWS Lambda function (`DVSA-ORDER-MANAGER`).
- **Tools Used:** Terminal/Git Bash (`curl` command), DVSA Web Interface.

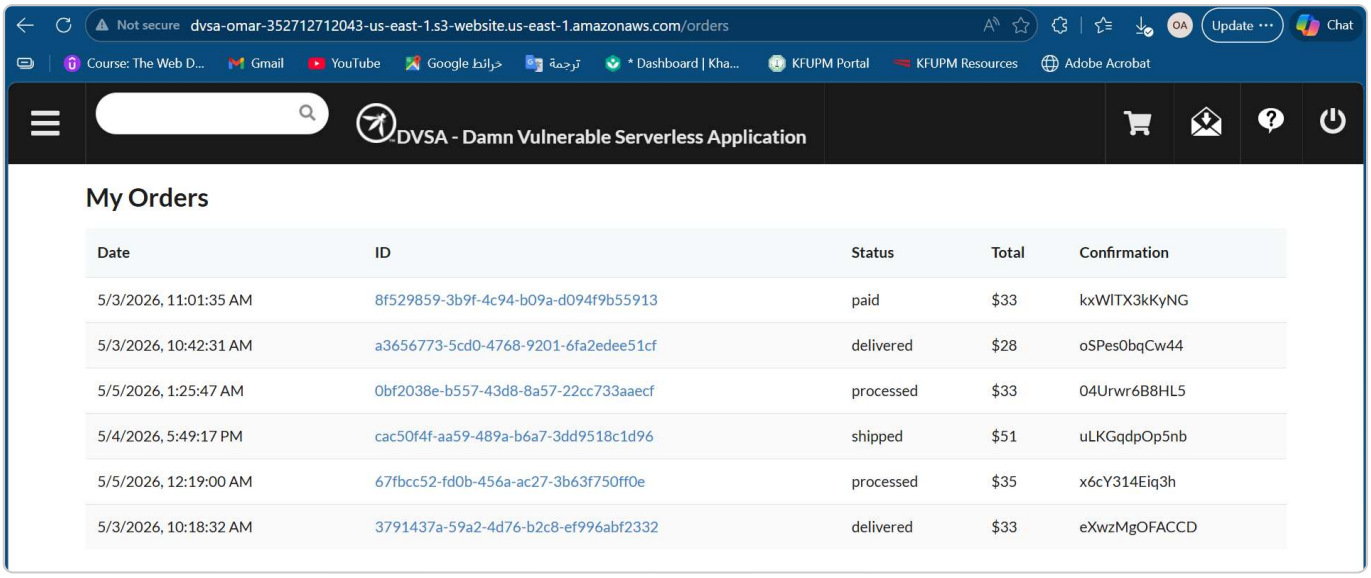
Part 4) Reproduction Steps

1. Identify the `/orders` API endpoint to target.
2. Obtain the `order-id` of the target order that you want to unlawfully alter.
3. Prepare a malicious POST request declaring the action as `admin-update`.
4. Provide the necessary JSON body defining the targeted `order-id` and applying an arbitrary modified `status` (e.g., "cancelled" or "shipped").
5. Submit the forged `curl` request carrying the malicious payload and observing the bypass.
6. Check the DVSA Website 'My Orders' section to confirm the order's status has been fraudulently updated.

Part 5) Evidence and Proof

Using the malformed `curl` command, an attacker submits the counterfeit request mimicking an administrator. Although an invalid header format may be logged occasionally, the payload execution still forces the backend to update the specified user's order status to the injected state without adequate role validation. The web application directly demonstrates the success of this exploit, reflecting the updated status values without any administrator involvement.



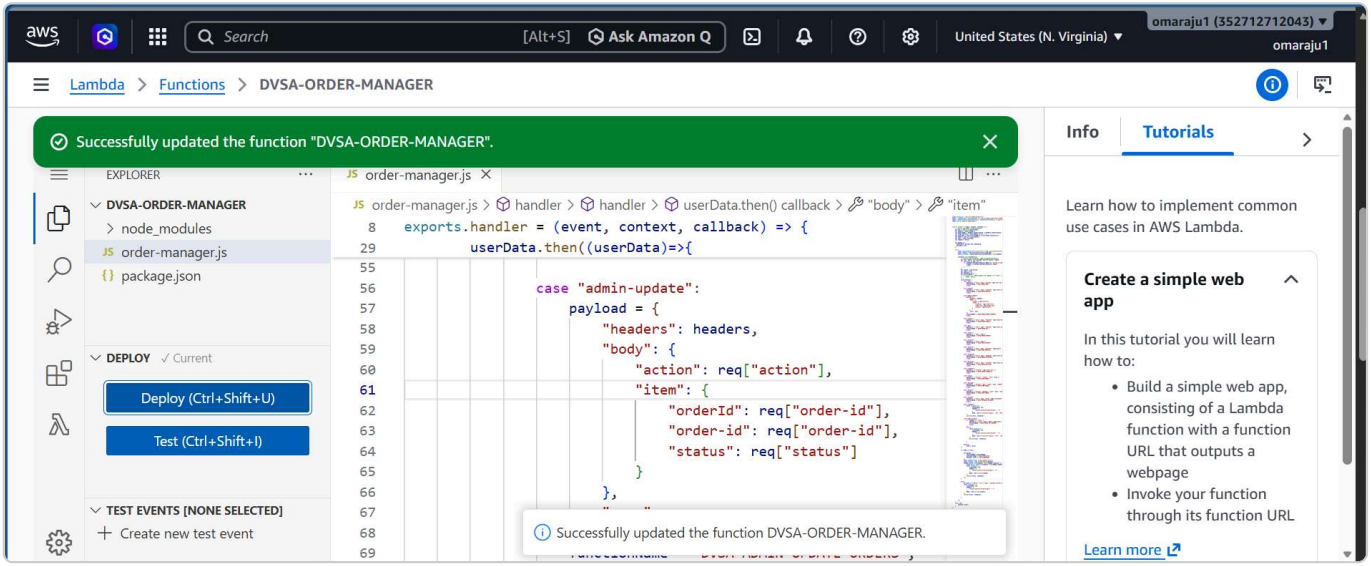


Part 6) Fix Strategy / Probable Mitigation

The fix should be applied in the `DVSA-ORDER-MANAGER` Lambda function. The core remediation handles verifying object level rights and implementing severe role-based checks. Before extracting or routing actions via `req["action"]`, the backend must retrieve user roles securely from validated token claims (not user-configurable body) and immediately reject the execution if the token owner lacks explicit administrative role tags.

Part 7) Code / Config Changes

The fix is targeted in the `DVSA-ORDER-MANAGER` source logic. Code must enforce robust authorization parameters. To properly secure it, changes verify user roles dynamically and ignore unauthenticated arbitrary `action` payload requests.



Part 8) Verification After Fix

After applying the fix, repeating the identical malicious payload test yields failure. The backend verifies credentials explicitly, throwing standard `403 Forbidden` or `401 Unauthorized` errors upon receiving the rogue request.

Part 9) Structured Operation and Security Analysis

Table A: Behavioral Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Access Control (BOLA/Privilege Escalation)	The `/orders` API should explicitly require administrative rights before performing state/object updates.	Bash CLI (curl), Browser, Lambda Handler Code.	Only administrators execute `admin-update` functionality. Standard user only fetches.	Arbitrary status injection modifies server backend data records successfully directly through REST paths.

Table B: Deviation Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Broken Access Control	The system accepts declarative role action states directly from untrusted input schemas without secure assertions.	Insecure Design / Broken Authorization.	Enforce role checks structurally in `DVSA-ORDER-MANAGER` controller cases.	After the fix, the injected request fails correctly as an unauthorized attempt.

Part 10) Takeaway / Lessons Learned

This exploitation was successful because the backend improperly trusted caller-provided states instead of server-verified token claims when establishing contexts. In Serverless APIs, failing to enforce explicit functional level and object level authorization enables attackers to elevate privileges laterally or vertically. This underscores the necessity of implementing absolute Zero-Trust validations rigorously at all data alteration endpoints.